

The Read-Determinism Test: A Standalone Operational Procedure for Verifying Same-State Read Identity in Coordination Knowledge Substrate Deployments

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the **read-determinism test** — the operational procedure that verifies the read-determinism guarantee of the CKS determinism contract — as a standalone test specification with independent operational content, separable from the cell-behavior-determinism test, the change-addressability test, the conflict-preservation test, and the substrate-as-source-of-truth test alongside which it composes within contract verification.

Abstract

The CKS determinism contract names five guarantees over substrate state, derived from commitments at §2.1, §3.1, §4.1, §6.2, §8.2, and §11.3 of the source paper. The first guarantee — read determinism — commits any CKS-coherent substrate to the property that two reads of identical substrate state, by any reader, through any cell that supports reading, yield identical content. A separate derivation note formalizes the contract as a unified specification, and a sibling note formalizes the test for the second guarantee (cell-behavior determinism). This note formalizes the test for the first guarantee — the read-determinism test — as a standalone procedure with independent operational content. It states what the test verifies, specifies the test as a sequence of operational steps, names pass and fail conditions at the substrate-content layer, enumerates the canonical anti-patterns the test detects, describes how the test integrates with deployment verification, and names the limits of what the test does and does not establish.

1. Why the read-determinism test needs to be formalized as standalone

Read determinism is the foundational guarantee of the determinism contract. Without it, the other four lose their operational reference: cell-behavior determinism, change addressability, conflict preservation, and substrate-as-source-of-truth all presuppose that substrate state reads as well-defined content rather than as a moving target. The source paper makes the commitment at §4.1 (the substrate is *deterministic*, *human-governed*, *auditable*) and at §11.3 (the substrate is the source of truth for what was decided and where contradictions remain) — commitments that are about reads as much as about writes, since a source of truth must yield the same answer to "what does it currently contain" across consecutive same-state reads, otherwise the source-of-truth status is decorative rather than operational.

Verifying the property in practice requires a procedure separable from the other four guarantees. Vendor caching introduces failure modes that are specifically read-side; distributed-substrate replication produces read-side drift during convergence; LLM-mediated retrieval introduces read-side summarization variance. None of these is detected by tests targeting cell-behavior determinism, addressability, or source of truth specifically. Without a standalone read-determinism test, deployments that satisfy the other four but fail read determinism go architecturally undiagnosed, and the failure surfaces later as inconsistency in dependent tests. Naming the test explicitly is what makes the demonstration auditable and gives implementers a citable reference for what their read-side verification must cover.

2. The architectural commitment under test

The test verifies the property the determinism contract names as its first guarantee:

Two reads of identical substrate state, by any reader, through any cell that supports reading, yield identical content.

Same-state reads, not all-time reads. Read determinism does not commit the substrate to producing identical content across all time. Substrate state changes under writes, and reads after writes are expected to differ. The commitment is narrower: between two reads with no intervening writes that affect the read's scope, the content read must be identical. The temporal qualifier — *same-state* rather than *same-time* — is the framing the source paper carries at §3.3 when it locates the substrate's commitments architecturally rather than procedurally.

Substrate content, not natural-language wrapping. What the substrate actually carries — entities, relationships, decisions, rationale, conflict states, provenance — must be identical across same-state reads. Natural-language wrapping that an LLM mediator may produce around a read is not bound by this test; the source paper's two-layer scope explicitly excludes binding LLM outputs as part of the determinism commitment.

Bounded variance is permitted, narrowly. The contract permits variance from operational sources that do not change substrate content — clock-stamp differences, network-jitter timing, identifiers generated outside substrate state. Variance that touches substrate content itself — different decisions, rationale, conflict states, or attribution — is what the test detects as failure. The test scope is precisely the substrate-content layer.

3. The test procedure

Step 1 — Establish a substrate state baseline. Identify the substrate state under test by selecting a defined scope — an addressable region of substrate content. Record the state at time t_1 by performing a read through the deployment's primary read path.

Step 2 — Re-read the same substrate state. At time t_2 , with no writes between t_1 and t_2 that would change content within the chosen scope, perform a second read through the same read path.

Step 3 — Compare the two reads at the substrate-content layer. The two recorded reads must be identical as substrate content. Differences in clock stamps, in identifiers generated outside substrate content, or in adjacent metadata not part of substrate state are not failures. Differences in any substrate content within the read scope are failures.

Step 4 — Cross-path verification. Where the deployment supports more than one read path — direct substrate read by a human exercising the inspect right, read by a cell consulting substrate state, read through a derived view — perform reads through each path with the same substrate state. Substrate content must be identical across paths. This step catches deployments where one path is deterministic and another is not.

Step 5 — Cross-vendor verification (where applicable). Where the deployment has migrated host environments under the tool-agnosticism commitment (§7.1, §7.4), perform a read after migration with the same substrate state established before. Substrate content must be identical pre- and post-migration. Read determinism is the read-side expression of what tool-agnosticism promises.

4. Test outputs

The test produces a binary pass/fail outcome at the substrate-content layer.

Pass conditions, conjunctively. All of the following must hold: Step 3 produced identical substrate content across consecutive same-state reads; Step 4 produced consistent substrate content across read paths; Step 5, where applicable, produced identical substrate content pre- and post-migration; and any variance observed is bounded to categories the contract permits.

Fail conditions, disjunctively. Any of the following constitutes failure: different substrate content despite no intervening writes (Step 3); inconsistent substrate content across read paths (Step 4); different substrate content across vendor environments (Step 5); or variance that includes substrate-content elements (different decisions, rationale, conflict states, or attribution).

A failed test diagnoses the deployment as not satisfying read determinism specifically, regardless of how it performs on the other guarantees. The diagnosis is precise: the read-side property is the failure point, and the anti-pattern named in §5 identifies the mechanism.

5. Anti-patterns the test detects

Vendor-cached read variance. Caching layers between readers and the substrate — content delivery networks, edge caches, in-memory stores — return content based on cache state rather than current substrate state. Two consecutive reads at cache-cold and cache-warm produce different content despite no intervening writes. Caches that may serve stale content as authoritative are inadmissible; caches that function only as performance layers below the substrate's authoritative read path do not violate the test.

Eventual-consistency drift. Distributed substrate deployments propagate writes across replicas asynchronously. During convergence, reads from different replicas, or from the same replica at different convergence states, return different content for the same logical state. The contract permits bounded latency in convergence; what it does not permit is reads during convergence returning content the converged state does not actually carry.

LLM-mediated read summarization. When the path from a reader to substrate content runs through an LLM that produces a summary or paraphrase rather than returning substrate content itself, the read inherits the LLM's non-determinism. The contract's two-layer scope excludes binding LLM outputs; the LLM cannot be a permissible read path. The AI-as-substrate-mediator commitment (§4.1, §4.2) permits the LLM to consult substrate content for adjacent purposes — drafting, lookup, search, navigation — but it does not

permit the LLM to *be* the read surface humans and cells use to observe substrate state.

Vendor-managed read variance. Some host environments perform A/B testing, content variation, or adaptive personalization on what they return for nominally identical reads. A substrate hosted on such an environment, where vendor variation operates on substrate content rather than only on adjacent presentation, fails read determinism. A/B testing is not forbidden in deployments using CKS substrates, but it cannot operate on substrate content itself without breaking the guarantee.

Background-process read interference. Background processes — re-indexing, replication, garbage collection, schema migration — can change read-side observable content during execution. The anti-pattern is not their existence, which is operationally necessary, but their effect on read-observable substrate content; properly architected, they operate without changing what reads return.

LLM-context-mediated reads. When reads are answered from the LLM's context window rather than from substrate state — the LLM having loaded prior content into context and now answering "what does the substrate contain" from context rather than re-reading — the read is determined by LLM context state, and two reads at different context-window states return different content even without substrate writes. This is the read-path-specific instance of what §6.2 names as *context rot* applied to coordination state. Treating it as a distinct anti-pattern is what makes it diagnosable at the read step rather than later.

6. Integration with deployment verification

The test is operationally meaningful at points in the deployment lifecycle where its results inform decisions. Five integration points cover the cases the source paper's commitments most directly engage.

Initial deployment validation. The test runs before a CKS deployment is activated for production coordination. Failure indicates the configured deployment does not satisfy the read-side guarantee; activation should not proceed without remediation, because every downstream test that reads substrate state will trust an unverified foundation.

Composition partner verification. When a CKS substrate is composed with another substrate or with an adjacent system, the test runs against the composed deployment. Composition is a common point of regression for read determinism specifically, because it often introduces caching, mediation, or projection layers that individually preserve content but jointly break read identity.

Vendor migration verification. When a deployment migrates host environments under the tool-agnosticism commitment, the test runs after migration with the same substrate state established before. Failure indicates the migration has not preserved read-side determinism. Satisfying the test is what makes tool-agnosticism a real property of a particular migration.

Caching-layer-update verification. When a deployment changes its caching configuration — adding a CDN, altering invalidation policy, introducing an in-memory cache, changing time-to-live policy — the test runs to verify the change has not introduced cache-induced variance. This is the most common operational point at which read-determinism failures surface, because caching changes are routinely treated as performance changes when they are also correctness changes for the read-side property.

Distributed-substrate convergence verification. For deployments using distributed substrate replication, the test runs periodically during steady-state operation. Reads that differ in substrate content during convergence indicate the replication mechanism violates the read-determinism guarantee, not merely that propagation is slow.

7. Limits of the test

Four limits deserve explicit naming, because each marks a property the test does not establish even when it passes.

The test does not verify cell-behavior determinism. Same-state reads being identical does not imply that cells executing under the same orchestration rules over the same substrate state produce equivalent decisions. That is the contract's second guarantee, tested by a sibling procedure. A deployment may pass read determinism while failing cell-behavior determinism — the substrate is read identically, but cells do different things with what they read.

The test does not verify change addressability. Read determinism verifies that what the substrate carries reads identically; it does not verify that changes are addressable to writers, rules, and rationale. That is the third guarantee — the path-retraceability commitment §3.1 imports — tested separately.

The test does not verify substrate-as-source-of-truth. Read determinism is a property of how substrate state is read once a deployment treats the substrate as the read target. It does not establish that the substrate is in fact where coordination state lives. A system can have deterministic reads over a substrate that is not the source of truth; the deterministic reads then merely confirm that whatever the substrate carries is read consistently, not that the substrate is what should be read.

The test does not verify content correctness or coverage. A deployment whose substrate is missing important coordination state, or whose substrate carries content stakeholders would judge incorrect, can pass the test fully. The test is about the read property over whatever the substrate carries, not about whether the substrate carries the right content.

8. One-sentence test

A CKS deployment satisfies read determinism if and only if two reads of identical substrate state, taken at times with no intervening substrate-content writes, through any read path the deployment supports, before and after any vendor migration the deployment has performed, return identical substrate content with all observed variance bounded to the categories the determinism contract permits.

9. Why naming the test as standalone matters

Read determinism is the most directly verifiable of the five contract guarantees, and the foundational one. Without it, none of the others is operationally checkable: the cell-behavior-determinism test reads substrate state, the addressability test reads substrate state, the conflict-preservation test reads substrate state, and the source-of-truth test reads substrate state. Each presupposes the read property the present test verifies. Treating it as a corollary of the contract risks leaving the foundational property untested and the dependent tests trusting an unverified foundation, surfacing read failures only as cascade effects.

A standalone test gives implementers a procedure separable from the contract as a whole, exercising the failure modes that vendor caching, eventual consistency, LLM mediation, vendor variance, background processes, and LLM context state most commonly introduce, and citable by deployments and auditors as the read-side gate the contract's other guarantees stand on top of.

The test pairs with the cell-behavior-determinism test: together they verify the two most directly checkable guarantees of the contract — that reads are stable for stable state, and that cells produce equivalent substrate writes for equivalent state-and-rules pairs. It precedes the provenance-completeness test (change addressability), the substrate-as-source-of-truth test, the conflict-preservation test, and the reproducibility test (which composes read determinism and addressability for replay verification).

Subsequent work that implements, extends, composes with, or argues against the CKS read-determinism commitment should use the read-determinism test in the sense formalized here. Subsequent work that uses a different procedure should name the difference; subsequent work that omits the test should name the omission as a known scope reduction in its claim to CKS coherence.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Read-Determinism Test: A Standalone Operational Procedure for Verifying Same-State Read Identity in Coordination Knowledge Substrate Deployments*. May 7, 2026. ORCID: 0009-0004-8065-3235.